

Project Work in Mathematical Finance

Laurea Magistrale in Banca e Finanza - Quantitative Finance 2025/26
Group: Scion Capital

Edoardo Anfelli

Lucia Ceola

Matteo Pasqualini

May 16, 2026

Exercise 1 – Dividend class hierarchy

We created a new package `net.finmath.tree.assetderivativevaluation.dividends` containing a complete hierarchy of interfaces and concrete classes for dividend modelling. The design follows the structure suggested in the assignment while adding some convenience features.

At the root of the hierarchy sits `DividendModel`, a marker interface that serves as the common super-type for any dividend specification. Directly below it, the interface `MultiplicativeDividendModel` declares the two core methods that every multiplicative dividend model must provide:

- `getCumulativeDividendFactor(double time)` – returns the cumulative factor $D(t)$ such that $S_t = S_t^{\text{no-div}} \cdot D(t)$;
- `getForwardDividendFactor(double time1, double time2)` – returns the forward factor $D(t_1, t_2) = D(t_2)/D(t_1)$, useful for step-by-step computations.

Additionally, the interface provides a default method `snapToGrid( $\Delta t$ )` that remaps discrete dividend times to the first grid point $\geq t_{\text{div}}$, via $\tilde{t} = \lceil t_{\text{div}}/\Delta t \rceil \cdot \Delta t$. This is essential to preserve the uniform time discretisation on which the risk-neutral probability formulas depend. The snapping is **not** left to the user: each tree model constructor automatically calls `dividendModel.snapToGrid(getTimeStep())` right after storing the dividend model, so that every discrete event sits exactly on a grid point before the tree is built. For continuous dividend models (`ContinuousDividendYield`, `NoDividends`) the default implementation is a no-op.

Each individual dividend payment is represented by the interface `DividendEvent`, which declares `getTime()` and `getDelta()` (the proportional amount $\delta \in [0, 1)$). The four concrete implementations are:

- **ProportionalDividend** – a single proportional dividend event. The stock drops by a factor $(1 - \delta)$ at time t_{div} : $D(t) = (1 - \delta)$ for $t \geq t_{\text{div}}$, and $D(t) = 1$ otherwise. This class implements both `MultiplicativeDividendModel` and `DividendEvent`. Its `snapToGrid` replaces the internal time with $\lceil t_{\text{div}}/\Delta t \rceil \cdot \Delta t$.
- **DiscreteProportionalDividends** – handles multiple dividend events. It stores a sorted list of `DividendEvent` objects and computes $D(t) = \prod_{t_k \leq t} (1 - \delta_k)$. The constructor accepts parallel arrays of times and deltas for convenience. Its `snapToGrid` iterates over all events and rebuilds each one with the snapped time.
- **ContinuousDividendYield** – models a constant continuous dividend yield q , so that $D(t) = e^{-qt}$ and $D(t_1, t_2) = e^{-q(t_2 - t_1)}$. No grid snapping is needed since this is a continuous-time specification; the default no-op applies.
- **NoDividends** – a trivial implementation returning $D(t) = 1$ everywhere. This class is used as default in the model constructors to preserve backward compatibility.

Exercise 2 – Models with dividend support

All three tree models (`CoxRossRubinsteinModel`, `JarrowRuddModel`, `BoyleTrinomial`) were extended to support dividends. In each case we added new constructors that accept a `MultiplicativeDividendModel` parameter, while the original constructors remain unchanged and internally delegate to the new ones using `new NoDividends()`. Every dividend-aware constructor calls `dividendModel.snapToGrid(getTimeStep())` immediately after storing the model.

The key design principle is that **proportional and discrete proportional dividends** do not alter the transition probabilities. The up/down factors (or up/mid/down for the trinomial) and the risk-neutral probabilities remain identical to the no-dividend case. At each level k of the tree, the spot values are simply multiplied by $D(k \cdot \Delta t)$ in the `buildSpotLevel(k)` method.

Continuous dividend yield q , on the other hand, modifies the risk-neutral drift and therefore requires adjustments to the probabilities:

Model	Adjustment with continuous yield q
CRR	The growth factor becomes $e^{(r-q)\Delta t}$ instead of $e^{r\Delta t}$, yielding $p = (e^{(r-q)\Delta t} - d)/(u - d)$. The up/down factors $u = e^{\sigma\sqrt{\Delta t}}$, $d = 1/u$ are unchanged.
Jarrow–Rudd	The drift is adjusted: $\mu^* = (r - q - \frac{1}{2}\sigma^2)\Delta t$, giving $u = e^{\mu^* + \sigma\sqrt{\Delta t}}$, $d = e^{\mu^* - \sigma\sqrt{\Delta t}}$. The probabilities remain $p_u = p_d = 1/2$.
Boyle	The effective rate becomes $(r - q)$, so $R = e^{(r-q)\Delta t}$ and $M_2 = e^{2(r-q)\Delta t + \sigma^2\Delta t}$. The trinomial probabilities p_u , p_d , p_m are recomputed from the modified R and M_2 .

The continuous yield is detected at construction time via `instanceof ContinuousDividendYield`, and an `effectiveGrowth` field is computed once. The spot levels are also multiplied by $D(k \cdot \Delta t) = e^{-q \cdot k \Delta t}$ via the same `getCumulativeDividendFactor` mechanism used for discrete dividends, ensuring a uniform code path.

Exercise 3 – Numerical tests on dividends

We created the test class `DividendOptionPricingTest` with four test methods. All tests use $S_0 = 100$, $r = 5\%$, $\sigma = 20\%$, $K = 100$.

Test 1: American call divergence (CRR, $T = 2$, 200 steps). Without dividends, the American and European call prices coincide exactly at 16.138. After introducing a 10% proportional dividend at $t = 1$, the American call rises to 11.694 while the European call drops to 9.904, a difference of **1.79**. This confirms that dividends make early exercise of the American call optimal in some states.

Test 2: Discrete proportional dividends (CRR, $T = 3$, 300 steps, 5% at $t = 1, 2$).

	European Call	American Call	European Put	American Put
CRR	14.316	14.863	10.137	11.686

Both American prices are strictly above their European counterparts, and the put early-exercise premium (1.55) is larger than the call one (0.55), consistent with the fact that put early exercise is already valuable without dividends.

Test 3: Continuous yield across all models ($T = 1$, 200 steps, $q = 3\%$). CRR and Jarrow–Rudd produce nearly identical European call prices (≈ 7.084 , difference < 0.001). Boyle gives 7.296, showing somewhat slower convergence at 200 steps, which is expected for the trinomial model. All American calls are strictly above their European counterparts.

Test 4: Cross-model consistency (JR vs. Boyle, $T = 2$, $K = 110$, $\sigma = 25\%$, 5% div. at $t = 1$). European put prices agree within 0.01 (JR: 16.042, Boyle: 16.033). American puts are 17.882 (JR) and 17.876 (Boyle), confirming convergence across models.

Exercise 4 – Barrier options

We implemented `BarrierOption` in the products package, supporting all eight barrier types (Down/Up $\times$ In/Out $\times$ Call/Put) plus an optional rebate parameter. The class extends `AbstractNonPathDependentProduct` and overrides the `getValues` method.

OUT barriers are priced via backward induction with knockout logic. At the terminal level, the payoff is computed as usual (e.g. $\max(S - K, 0)$ for a call), but it is set to the discounted rebate at any node where the barrier has been breached. During backward induction, at each intermediate level k , we check each node: if $S_k \leq H$ (for Down-Out) or $S_k \geq H$ (for Up-Out), the option value at that node is replaced by the discounted rebate; otherwise, the standard risk-neutral conditional expectation applies.

IN barriers are computed via the in–out parity identity: $V_{\text{In}} = V_{\text{European}} - V_{\text{Out}}$. When an IN barrier is requested, the code internally prices the corresponding European option and the corresponding OUT barrier with the same parameters, then returns their difference. This avoids duplicating the backward-induction logic and is numerically exact (no approximation beyond the tree discretisation itself).

We verified the parity identity numerically: with $S_0 = 100$, $K = 100$, $H = 90$ (Down barrier), $T = 1$, 500 steps on CRR, we obtained Down-In Call = 1.712, Down-Out Call = 8.742, European Call = 10.454, and In + Out = 10.454, confirming the identity to machine precision.

Exercise 5 – Barrier prices vs. closed-form formulas

We compared tree prices against the Black–Scholes analytical values provided in the class `it.univr.fima.barrieroptionsformulas.BarrierOptions` (the corrected version supplied with the template, not the one in `net.finmath.functions`). Test parameters: $S_0 = 100$, $r = 5\%$, $\sigma = 20\%$, $T = 1$, $K = 100$, rebate = 0, $q = 0$, **500 time steps**.

Barrier option	Analytical	CRR	JR	Trinomial	Max $ \Delta $
Down-Out Call ($H = 90$)	8.665	8.742	8.828	8.953	0.288
Down-In Call ($H = 90$)	1.785	1.712	1.623	1.496	0.289
Up-Out Call ($H = 120$)	1.176	1.290	–	–	0.114
Up-In Call ($H = 120$)	9.275	9.164	9.148	9.118	0.157
Down-Out Put ($H = 90$)	0.151	0.164	0.179	0.197	0.046
Down-In Put ($H = 90$)	5.422	5.413	5.395	5.375	0.048
Up-Out Put ($H = 120$)	5.360	5.392	5.382	5.396	0.036
Up-In Put ($H = 120$)	0.213	0.185	0.192	0.176	0.038

All tree prices fall within 0.3 of the analytical value, and CRR consistently provides the closest match. The residual discrepancy is expected: barrier options converge more slowly than vanilla options because the tree monitors the barrier only at discrete time steps, introducing a systematic bias relative to the continuous-monitoring assumption of the Black–Scholes formula. For instance, the Down-Out Call CRR error drops from 0.076 at 500 steps to below 0.02 at 2000 steps. The Down-In and Down-Out errors are symmetric (≈ 0.29 for the trinomial) because the in–out parity mirrors any OUT error into the IN price.